REFACTORING
EJERCICIO 5 PRODUCTOS.

Se cuenta con un sistema que maneja una jerarquía de productos que pueden ser estadías de hoteles o alquileres de autos.
En carpeta *material adicional* puede descargar un proyecto con las clases implementadas.
Observe que el proyecto tiene test unitarios que verifican el funcionamiento correcto de la aplicación.
A continuación se muestra un extracto del código:

HotelStay.java

```
1  public class HotelStay extends Product {
2      public double cost;
3      private TimePeriod timePeriod;
4      private Hotel hotel;
5
6      public HotelStay(double cost, TimePeriod timePeriod, Hotel hotel) {
7          this.cost = cost;
8          this.timePeriod = timePeriod;
9          this.hotel = hotel;
10     }
11
12     public LocalDate startDate() {
13         return this.timePeriod.start();
14     }
15
16     public LocalDate endDate() {
17         return this.timePeriod.end();
18     }
19
20     public double priceFactor() {
21         return this.cost / this.price();
22     }
23
24     public double price() {
25         return this.timePeriod.duration() * this.hotel.nightPrice() * this.hotel.discountRate();
26     }
27 }
28
```

CarRental.java

```java
public class CarRental extends Product {
    public double cost;
    private TimePeriod timePeriod;
    private Company company;

   public CarRental(double cost, TimePeriod timePeriod, Company company) {
        this.cost = cost;
        this.timePeriod = timePeriod;
        this.company = company;
    }

    public LocalDate startDate() {
        return this.timePeriod.start();
    }

    public LocalDate endDate() {
        return this.timePeriod.end();
    }

    public double price() {
        return this.company.price() * this.company.promotionRate();
    }

    public double cost() {
        return this.cost;
    }
}
```

Tareas:

1.La variable “cost” está declarada como pública, lo que rompe el encapsulamiento de la clase.
Utilice el refactoring Encapsulate Field y describa brevemente los pasos llevados a cabo.
Verifique que los tests provistos sigan funcionando.
<u>Discuta con un ayudante:</u>
a. ¿Es correcto modificar alguno de los tests para que el código refactorizado funcione?
b. En caso de qué el test falle, ¿qué situación está representando este test?

2.Use el refactoring Rename Field en el método priceFactor(), para que la variable “cost” se pase a llamar “quote”.
Verifique que los tests provistos sigan funcionando.
<u>Discuta con un ayudante:</u>
a.¿en este caso, es necesario modificar alguno de los tests para que el código refactorizado funcione?

3. Se quiere aplicar el refactoring **Pull Up Method** para subir los métodos startDate() y endDate() a la superclase Product.
¿Es posible hacerlo en el código anterior?
Justifique su respuesta basándose en las precondiciones del refactoring vistas en la

teoría y en el libro de Refactoring de Martin Fowler.

4.Mencione y aplique los refactorings necesarios para poder hacer Pull Up Method.

5.Aplique Pull Up Method para subir los métodos a la superclase Product.
Verifique que los tests provistos sigan funcionando.

6.Observe los métodos:
price() en CarRental.java (línea 21) y en HotelStay.java (línea 25).
Identifique los Code Smell presentes.
Luego aplique los refactoring correspondientes para eliminarlos.
Verifique que los tests provistos sigan funcionando.